알고리즘2 시험 당일 압축요약

개념의 구분 → 상태·점화식 → 코드의 경계 조건. 처음 배우기 위한 전체 강의 노트는 학습실에 있고, 이 문서는 시험 직전 복습용이다.

범위는 공지의 **7주차까지(Lec.00–06)**. 주관식 단답형·코드 채우기, 코드는 의사코드로 채점. **10월 29일 11:00–12:00 / IT-5호관 245호**. 공학용 계산기 외 전자기기는 사용하지 못한다.

10분 경로: SST → Master → QUADTREE → Huffman → Prim → 마지막 암기 목록. **30분 경로:** 여기에 DP 갱신 방향·배낭·LIS/KLIS·POLY를 더한다. 출제 예고가 전체 범위를 대체하지는 않는다.

코드는 학습실 구현을 압축/선별한 설명용 구현이다. 원본 과제의 입출력 코드와 구분한다. PDF 쪽수는 표지 포함 실제 쪽수다. 중요한 영어 용어는 해당 개념 안에서 확인한다.

가로 2단 · 본문은 HTML과 공유 · 코드 왼쪽의 ↳는 인쇄용 줄바꿈 표시

LEC.00 · 문제 조건과 불변식

정답·종료·효율을 구분한다

Lec.00 pp. 2–11 · Lec.00 pp. 12–14 · Lec.00 pp. 15–21 · Lec.01 pp. 2–7

Correctness(정확성)는 허용 입력 전체에서 원하는 답을 내는 성질이다. **Finiteness(유한성)**는 끝나는가, 효율성은 얼마나 많은 시간·공간을 쓰는가이다. 종료하면서 오답을 낼 수도 있다.

알고리즘의 조건은 **Input 0개 이상 / Output 1개 이상 / Finiteness / Definiteness(명확성) / Effectiveness(실행 가능성)**이다. “열심히 생각한다”는 단계는 명확한 연산이 아니다.

루프 불변식은 **초기 성립 → 한 번 실행해도 유지 → 종료 때 정답 연결** 순서로 쓴다.

`s += i` 직전은 $s = \frac{i(i-1)}{2}$, 직후는 $s = \frac{i(i+1)}{2}$이다. 전체 정확성을 보이려면 종료성도 필요하다.

Collatz 3n+1은 모든 양의 정수에서 종료하는지 미해결이다. 기출은 유한성을 겨냥하지만 “어떤 입력은 반드시 무한 반복한다”가 증명된 것은 아니다. 강의의 3n−1 예와 섞지 않는다.

코드 답안은 의사코드로 채점한다. 문법 장식보다 **종료값·상태·반복 범위·부작용 복구**를 확인한다. 한 예제 통과는 정확성 증명이 아니다.

COINFLIP · 짝홀성은 가능 여부, 거리는 최소 횟수

Lec.00 pp. 22–30

인접한 두 동전을 같이 뒤집으면 T 개수는 −2·0·+2만큼 변한다. **T 개수의 parity(짝홀성)는 불변**이다. 모두 H인 목표는 T가 0개이므로 처음 T가 홀수이면 불가능, 짝수이면 가능하다.

최소 횟수는 왼쪽부터 T 두 개씩 짝지어 **위치 차이의 합**이다. `THHT`는 위치 0과 3 → **3회**. `THHT → HTHT → HHTT → HHHH`. T가 2개라는 이유로 1회라고 쓰지 않는다.

반환 −1은 불가능, 0은 이미 목표이다. **시간 $O(N)$, 추가 공간 $O(1)$.**

구현 핵심

```
def coinflip2(coins):
    unmatched = -1
    total = 0
    for i, state in enumerate(coins):
        if state == "T":
            if unmatched == -1:
                unmatched = i
            else:
                total += i - unmatched
                unmatched = -1
    return -1 if unmatched != -1 else total
```

LEC.01 · 복잡도와 구간 문제

Big-O와 P/NP · 무엇을 세는지 먼저 쓴다

Lec.01 pp. 8–12

구분	답에 쓸 핵심
O / Ω / Θ	각각 점근 상한 / 하한 / 같은 차수의 양쪽 경계
최악·평균·최선	어느 입력에 대해 비용을 세는가. O 자체가 “최악”은 아니다
P	다항 시간에 **푸는** 결정 문제
NP	받은 증거를 다항 시간에 **검증하는** 결정 문제. $P \subseteq NP$, $P = NP$는 미해결
NP-hard / NP-complete	모든 NP 문제에서 다항 시간 환원이 가능 / 그중 NP에도 속함

성장 순서: $1 < \log n < n < n\log n < n^2 < 2^n < n!$. n←n/2 반복은 $\Theta(\log n)$, 삼각 반복 $\sum_{i=0}^{n-1} i$는 $\Theta(n^2)$.

중첩 반복이라고 항상 $\Theta(n^2)$는 아니다. 스택에서 각 항목을 한 번 넣고 빼면 총 $O(n)$이다.

배열·캐시뿐 아니라 **재귀 스택**도 추가 공간이다. 비교 정렬의 최악 하한은 $\Omega(n\log n)$; 단순 Quicksort의 최악은 $\Theta(n^2)$, 무작위 피벗의 기대 시간은 $O(n\log n)$이다. (n−C)!와 n!는 고정 C>0이어도 같은 Θ로 바꾸지 않는다.

FESTIVAL · 누적합 규약을 섞지 않는다

Lec.01 pp. 13–31

길이 **L 이상**인 연속 구간의 **평균 최솟값**이다. 정확히 L일 또는 합의 최솟값이 아니다. 아래는 `P[0] = 0`이고 **[i,j)** 구간을 사용한다.

누적합 규약	구간 합	길이
앞에 0: P[t]=A[0:t]의 합	[i,j)의 합 = P[j]−P[i]	j−i
강의·과제: 마지막에 0, psum[−1]=0	[i,j]의 합 = psum[j]−psum[i−1]	j−i+1

`[3,1,2,3,1,2]`, L=3에서 [1, 2, 3, 1]의 평균은 **7/4=1.75**. 3일만 보면 놓친다. $O(N^3)$ **재합산 →** $O(N^2)$ **누적 갱신/누적합**; 구간 수 자체는 여전히 $O(N^2)$이다. 아래 구현 추가 공간 $O(N)$.

구현 핵심

```
def festival(cost, L):
    n = len(cost)  # 1 <= L <= n
    P = [0] * (n + 1)
    for i in range(n):
        P[i + 1] = P[i] + cost[i]
    best = float("inf")
    for i in range(n - L + 1):
        for j in range(i + L, n + 1):
            best = min(best, (P[j] - P[i]) / (j - i))
    return best
```

MAXSUM · 빈 구간 허용 여부가 초기값을 바꾼다

Lec.01 pp. 32–44

cur는 현재 원소에서 **끝나는 비어 있지 않은 구간**의 최대합이다. 새로 시작하는 x와 앞 구간을 잇는 `cur + x`를 비교한다. `best = 0`이면 전체 답에는 빈 구간을 허용한다.

[−5,−2,−7] → **강의 규약은 0**, 비어 있지 않아야 한다면 −2이다. 그 경우 best를 −∞ 또는 첫 원소로 시작한다.

완전탐색 $O(N^3)$ **→ 누적합** $O(N^2)$ **→ 분할정복** $O(N\log N)$ **→ Kadane** $O(N)$. 분할정복은 왼쪽·오른쪽·중앙을 가로지르는 최대합의 세 경우이다. 아래 추가 공간 $O(1)$.

구현 핵심

```
def maxsum(A):
    cur = best = 0
    for x in A:
        cur = max(cur, 0) + x
        best = max(best, cur)
    return best
```

LEC.02 · 완전탐색과 상태 공간

재귀 조합 · 선택을 바꿨으면 복구한다

Lec.02 pp. 2–14

재귀는 **종료 조건 → 다음 선택 → 상태 변경 → 재귀 → 복구** 순서다. 아래 코드는 **0..n−1** 중 r개를 고른다. 다음 후보는 `i + 1`부터 시작하므로 순서만 다른 중복이 없다. 선택한 개수가 r이면 **복사본**을 저장한다.

순열은 순서가 다르면 다른 답이고, 조합은 같은 집합이면 같은 답이다. 답을 전부 출력하는 비용도 센다: 조합 출력은 대략 $O(r \cdot C(n,r))$, 스택·현재 선택은 $O(r)$, 저장한 결과는 별도다.

인자에서 리스트를 공유하면 append 뒤 pop이 필요하다. `chosen + [i]`로 새 리스트를 넘기는 방식은 복구 대신 복사 비용을 지불한다.

같은 재귀 원리의 예: **Hanoi**는 n−1개 이동→가장 큰 원판→n−1개 이동이라 이동 횟수 $2^n - 1$, 호출 스택 $O(n)$이다. **행렬곱**은 `C[i][j] += A[i][k] * B[k][j]`이고 k가 끝나면 다음 j, j가 끝나면 다음 i로 간다. 반복을 재귀로 바꾸어도 $N \times K \times M$개의 곱셈이라는 일의 양은 줄지 않는다.

구현 핵심

```
def combinations(n, r):
    out, chosen = [], []
    def search(start):
        if len(chosen) == r:
            out.append(chosen[:])  # 복사해 보관
            return
        for i in range(start, n):
            chosen.append(i)
            search(i + 1)
            chosen.pop()  # 다른 가지를 위한 복구
    search(0)
    return out
```

CLOCKSYNC · 리프·전체 호출·깊이는 서로 다르다

Lec.02 pp. 15–26

출제 예고: SST 크기 계산. 루트 깊이 0, 스위치 d개, 스위치당 0·1·2·3회인 **4분기 완전 트리**를 센다.

물어보는 값	일반식	d=10
리프 수	4^d	4^{10} = 1,048,576
전체 노드·함수 호출 수	$\sum_{k=0}^{d} 4^k = \frac{4^{d+1}-1}{3}$	1,398,101
최대 깊이 / 동시에 쌓인 호출 수	d / d+1	10 / 11

가지치기 없는 기본 DFS의 수이며 개선 풀이의 탐색량과 섞지 않는다. 아래 `linked[s]`는 문제에서 제공한 **s번 스위치와 연결된 시계 인덱스 목록**이다. 입력 시간을 `(hour // 3) % 4`로 바꾸면 12시는 0이다.

search(s)는 s 이후 스위치로 맞추는 최소 추가 누름 수이다. **재귀를 먼저, push를 나중에** 하므로 cnt = 0도 조사한다. 루프가 끝나면 push가 4회 실행되어 입력 시계가 복구된다. 시간 $O(4^d)$, 스택 $O(d)$; 시계·연결 개수를 고정 상수로 본다.

구현 핵심

```
def clocksync(clocks, linked):
    INF = float("inf")
    def push(s):
        for c in linked[s]:
            clocks[c] = (clocks[c] + 1) % 4
    def search(s):
        if s == 10:
            return 0 if not any(clocks) else INF
        best = INF
        for cnt in range(4):
            best = min(best, cnt + search(s + 1))
            push(s)
        return best
    answer = search(0)
    return -1 if answer == INF else answer
```

CLOCKSYNC 개선 · 의존 순서대로 강제한다

Lec.02 pp. 27–38

시계 8에는 스위치 4만 영향을 준다. 현재 값이 2면 스위치 4는 **(−2)%4=2회**로 결정된다. 시계 11→스위치 1, 시계 13→스위치 9도 먼저 결정할 수 있다.

현재 시계 상태로 횟수 결정 → 연결된 모든 시계에 반영 → 다음 결정을 반복한다. 처음 값으로 모든 횟수를 한꺼번에 계산하지 않는다. 아래 순서는 이 강의 연결 표에만 유효하다. 마지막 전체 검사를 해야 모순을 거른다.

처음 세 스위치만 고정하고 나머지를 탐색하면 리프는 4^7이다. 개선 설명을 “시험 제외”로 해석하지 않는다.

구현 핵심

```
def determined(clocks, linked):
    sequence = [(8, 4), (11, 1), (13, 9), (6, 3), (10, 2),
                (7, 7), (4, 8), (1, 0), (3, 6), (0, 5)]
    total = 0
    for clock, switch in sequence:
        count = (-clocks[clock]) % 4
        for other in linked[switch]:
            clocks[other] = (clocks[other] + count) % 4
        total += count
    return -1 if any(clocks) else total
```

BOARDCOVER · 첫 빈칸 하나만 잡는다

Lec.02 pp. 39–53

0은 빈칸, 1은 막힘/이미 채움. L자 3칸 조각으로 덮는다. **빈칸 수가 3의 배수**가 아니면 답은 0이지만, 배수라는 조건만으로 배치가 존재하지는 않는다.

가장 위·왼쪽의 첫 빈칸을 고정하고 그 칸을 덮는 **네 모양**만 시도한다. 아무 빈칸이나 다음 대상으로 고르면 같은 배치를 다른 순서로 중복 센다. **빈칸 없음의 반환은 1**: 완성된 배치 한 가지다.

아래는 배치 전에 세 칸을 모두 검사하여 성공한 배치만 변경·복구한다. 강의의 `place(+1)`은 실패해도 일부 칸이 변할 수 있으므로 **그 버전에서는 성공 여부와 무관하게** `place(-1)`**을 호출**한다. 두 구현 규칙을 섞지 않는다. E개 빈칸에 대해 가지 수 상한 $4^{E/3}$, 각 호출에서 빈칸 찾기 $O(HW)$가 든다.

구현 핵심

```
SHAPES = [((0, 0), (1, 0), (0, 1)), ((0, 0), (0, 1), (1, 1)),
          ((0, 0), (1, 0), (1, 1)), ((0, 0), (1, 0), (1, -1))]

def cover(board):  # 0: 빈칸, 1: 막힘/이미 채움
    H, W = len(board), len(board[0])
    pos = next(((y, x) for y in range(H) for x in range(W)
                if board[y][x] == 0), None)
    if pos is None:
        return 1  # 완성된 배치 한 가지
    y, x = pos
    total = 0
    for shape in SHAPES:
        cells = [(y + dy, x + dx) for dy, dx in shape]
        if all(0 <= a < H and 0 <= b < W and board[a][b] == 0
               for a, b in cells):
            for a, b in cells: board[a][b] = 1
            total += cover(board)
            for a, b in cells: board[a][b] = 0
    return total
```

BOGGLE · 위치만으로는 상태가 부족하다

Lec.02 pp. 55–58

상태는 **(y,x,k)**: 그 칸에서 `word[k:]`를 만들 수 있는가. 범위 밖/글자 불일치 → False, 마지막 글자 일치 → True, 나머지는 **8방향 + k+1**이다.

이 수업 규칙에서는 **칸 재방문이 가능**하다. 이웃 A,B로 ABA를 찾을 수 있다. 같은 칸에서 이동 없이 두 글자를 읽을 수는 없다. 따라서 `(dy, dx) = (0, 0)`을 빼며, visited로 재방문을 막지 않는다.

길이 L, H×W 보드의 단순 DFS 상한 $O(HW \cdot 8^L)$, 스택 $O(L)$. 메모를 쓴다면 키에도 k가 있어야 한다. 추가 실습 권장이라는 말은 시험 제외가 아니다.

구현 핵심

```
def boggle(board, word):  # word는 비어 있지 않음
    H, W = len(board), len(board[0])
    def has(y, x, k):
        if not (0 <= y < H and 0 <= x < W): return False
        if board[y][x] != word[k]: return False
        if k == len(word) - 1: return True
        return any(has(y + dy, x + dx, k + 1)
                   for dy in (-1, 0, 1) for dx in (-1, 0, 1)
                   if dy != 0 or dx != 0)
    return any(has(y, x, 0) for y in range(H) for x in range(W))
```

LEC.03 · 분할정복과 재귀식

분할정복 · 같은 부분문제를 두 번 부르지 않는다

Lec.03 pp. 2–13

빠른 합 $S(n) = 1 + \cdots + n$에서 홀수는 $S(n-1) + n$, 짝수는 $2S(n/2) + (n/2)^2$이다. 거듭제곱은 절반 지수를 **한 번만 계산해 저장**한 뒤 제곱한다.

`power(A, m // 2)`를 두 번 호출하면 $2T(m/2) + O(1) = \Theta(m)$, 한 번만 호출하면 $T(m/2) + O(1) = \Theta(\log m)$이다. 고정 크기 행렬 곱을 상수 비용으로 본 설명이며 임의 차원 행렬 곱 비용은 별도다.

아래 `multiply`는 곱셈 함수, `identity`는 항등원(스칼라 1 또는 항등행렬)이다. **지수 0의 답은 항등원**. 시간·스택 $O(\log m)$.

빠른 합

```
def fast_sum(n):  # n >= 1
    if n == 1: return 1
    if n % 2: return fast_sum(n - 1) + n
    return 2 * fast_sum(n // 2) + n * n // 4
```

반환값을 저장하는 거듭제곱

```
def power(A, m, multiply, identity):
    if m == 0: return identity
    if m % 2: return multiply(power(A, m - 1, multiply, identity), A)
    half = power(A, m // 2, multiply, identity)
    return multiply(half, half)
```

Master Theorem · 코드 → a,b,k → 차수

Lec.03 pp. 14–16

출제 방식 안내: 코드가 주어질 수 있다. $T(n) = aT(\frac{n}{b}) + \Theta(n^k)$에서 **a=재귀 호출 수, b=입력 축소 비율, k=재귀 밖 작업의 지수**이다. 로그 밑 b와 호출 수 a를 바꾸지 않는다.

비교	타이트한 차수	예
$a > b^k$	$\Theta(n^{\log_b a})$	$7T(n/2) + \Theta(n^2) \to \Theta(n^{\log_2 7})$
$a = b^k$	$\Theta(n^k \log n)$	$2T(n/2) + \Theta(n) \to \Theta(n \log n)$
$a < b^k$	$\Theta(n^k)$	$3T(n/2) + \Theta(n^2) \to \Theta(n^2)$

재귀 호출 **2번**, 크기 **n/2**, 바깥 반복 **n회**이면 a = 2, b = 2, k = 1이다. 별도 $T(n-1)$이나 서로 다른 부분문제 크기에는 이 표를 기계적으로 대입하지 않는다. O를 물으면 위 Θ를 O로 써도 되지만 타이트한 차수를 고른다.

가벼운 가짜 동전 하나가 있다는 조건에서는 저울 결과가 세 갈래라 최악 $\lceil \log_3 n \rceil$회로 후보를 좁힌다(8개는 3·3·2로 나눠 2회). 무거운지 가벼운지도 모르는 다른 문제에는 이 식을 그대로 쓰지 않는다.

QUADTREE 뒤집기 · 읽는 순서와 붙이는 순서는 다르다

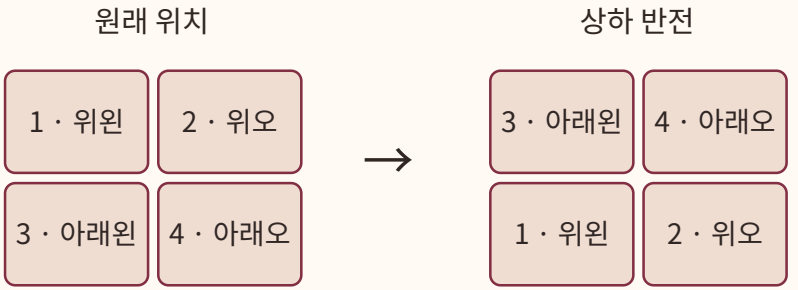
Lec.03 pp. 17–37 · Lec.03 pp. 38–42

출제 예고는 상하 뒤집기에 있다. `b` / `w`는 단색 잎, `x` 뒤는 **상좌 q1 → 상우 q2 → 하좌 q3 → 하우 q4** 네 자식이다. 한 자식 자체가 긴 트리일 수 있어 문자열을 같은 길이로 4등분하면 안 된다.

네 자식을 **원래 순서로 재귀적으로 읽고**, 반환은 `"x" + q3 + q4 + q1 + q2`이다. 재귀가 각 자식 내부도 이미 뒤집는다. 부모의 자리만 바꾸면 불완전한 반전이다. iterator 또는 읽기 인덱스는 자식 네 호출이 공유한다.

xbwwb → xwbbw → 다시 뒤집으면 xbwwb. n은 이미지 한 변이 아니라 여기서는 **인코딩 길이. 강의의 노드 방문 분석은** $O(n)$이다. Python 문자열을 새로 복사하는 비용까지 세면 높이 h에 대해 $O(nh)$, 균형 트리는 $O(n \log n)$, 매우 치우치면 $O(n^2)$까지 고려한다. 이미지의 픽셀을 채우는 디코딩 비용과 구분한다.

인코딩·디코딩은 표현 이해를 위한 구현 권고다. **그 두 구현 자체가 직접 출제 확정이라는 뜻은 아니다.** 자세한 구현은 학습실 링크에서 확인한다.



구현 핵심

```
def flip(encoded):
    tokens = iter(encoded)
    def read():
        head = next(tokens)
        if head in "bw": return head
        q1, q2, q3, q4 = read(), read(), read(), read()
        return "x" + q3 + q4 + q1 + q2
    return read()
```

FENCE · 중앙 걸침과 스택 너비를 구분한다

Lec.03 pp. 43–55 · Lec.03 pp. 56–65

답은 **왼쪽 내부 / 오른쪽 내부 / 중앙을 가로지름** 중 최대이다. 재귀 구간이 [left,right]이면 오른쪽은 **mid+1**부터. 중앙 걸침은 `[mid, mid + 1]`에서 시작해 더 높은 쪽으로 늘리며 **최소 높이 × (high−low+1)**를 갱신한다. 시간 $O(N \log N)$, 스택 $O(\log N)$.

단조 스택은 높이가 비감소하도록 인덱스를 보관한다. 더 낮은 판자 i를 만나 j를 pop한 **뒤의** 왼쪽 경계를 쓴다. 스택이 비면 너비 i, 아니면 **i−stack[−1]−1**. 양쪽 경계는 포함하지 않는다.

끝에 0 높이를 추가하면 남은 판자도 처리된다. 각 판자는 한 번 들어가고 나와 **시간** $O(N)$**, 공간** $O(N)$. 예: i=7, pop 뒤 왼쪽 경계=2, 높이=3이면 너비4·넓이12.

분할정복의 경계 확장

```
def fence(h, left, right):
    if left == right: return h[left]
    mid = (left + right) // 2
    best = max(fence(h, left, mid), fence(h, mid + 1, right))
    lo, hi = mid, mid + 1
    height = min(h[lo], h[hi])
    best = max(best, height * 2)
    while left < lo or hi < right:
        if hi < right and (lo == left or h[lo - 1] < h[hi + 1]):
            hi += 1
            height = min(height, h[hi])
        else:
            lo -= 1
            height = min(height, h[lo])
        best = max(best, height * (hi - lo + 1))
    return best
```

스위핑·단조 스택

```
def fence_stack(heights):
    h = heights + [0]  # 끝에서 남은 판자를 모두 처리
    stack, best = [], 0
    for i in range(len(h)):
        while stack and h[stack[-1]] >= h[i]:
            j = stack.pop()
            width = i if not stack else i - stack[-1] - 1
            best = max(best, h[j] * width)
        stack.append(i)
    return best
```

LEC.04 · 동적계획법의 기본

DP · 상태가 같으면 미래의 답도 같아야 한다

Lec.04 pp. 2–10

Optimal substructure(최적 부분구조)는 최적해가 부분문제의 최적해를 이용해 구성되는 성질, **Overlapping subproblems(중복 부분문제)**는 같은 부분문제를 여러 번 푸는 성질이다. 두 용어를 바꾸지 않는다.

상태의 뜻 → 점화식 → 기저 → 계산 순서 → 반환값 순서로 읽는다. Memoization은 필요한 상태를 재귀+캐시로 구하고, Tabulation은 의존하는 상태부터 표를 채운다. 비용은 **상태 수 × 한 상태의 전이 비용**으로 센다.

이항계수 $C(n,k) = C(n-1,k-1) + C(n-1,k)$, $C(n,0) = C(n,n) = 1$. 한 행으로 압축하면 j를 **큰 쪽에서 작은 쪽으로** 갱신해야 이전 행 값을 읽는다.

캐시의 −1은 미계산 표시이다. 답으로 0이 나올 수 있으면 0을 미계산으로 사용하지 않는다. 강의 지침에 맞춰 아래 요약 코드는 직접 표/딕셔너리를 사용한다. @cache만 붙이면 상태 설계나 메모리 문제가 해결되는 것은 아니다.

제약 조건 때문에 충분한 상태를 저장하기 어렵다면, 조건을 일찍 검사해 후보를 줄이는 **Backtracking(백트래킹)**이 적절할 수 있다. “제약이 있으면 무조건 DP 불가”는 아니다. 알파메틱처럼 문자↔숫자가 **일대일 대응**이면 사용한 숫자·선행 0 금지·각 식의 조건을 함께 관리한다.

한 행 이항계수

```
def choose(n, k):  # 0 <= k <= n
    dp = [1] + [0] * k
    for i in range(1, n + 1):
        for j in range(min(i, k), 0, -1):
            dp[j] += dp[j - 1]
    return dp[k]
```

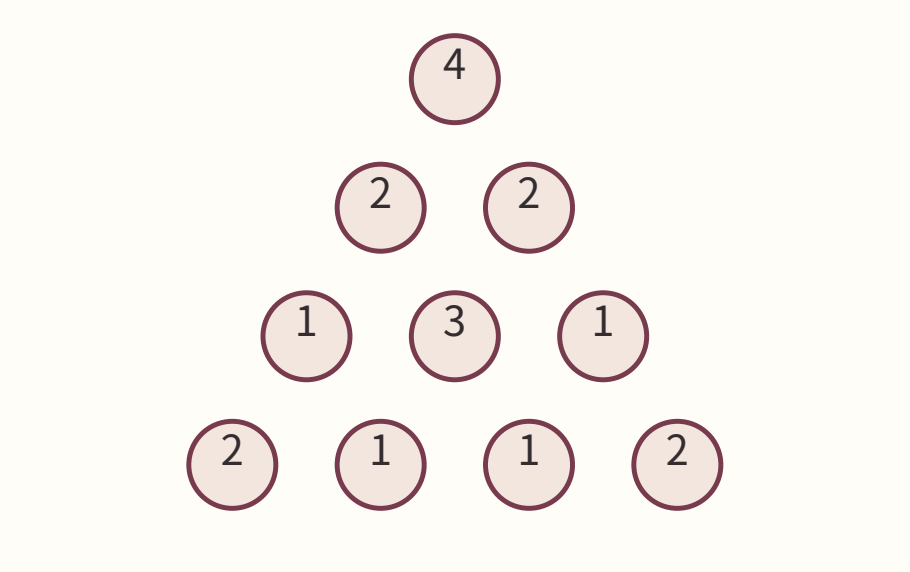
TRIANGLEPATH · 최댓값, 경로, 경로 수

Lec.04 pp. 11–30 · Lec.05 pp. 2–10

D[y][x]는 (y,x)에서 마지막 행까지 갈 수 있는 최대 합. $T_{y,x} + \max(D_{y+1,x}, D_{y+1,x+1})$이며 마지막 행이 기저다. 한 행 DP는 **아래→위, x는 왼쪽→오른쪽**으로 갱신한다. 오른쪽부터 하면 필요한 아래 행 값이 사라진다.

경로 복원은 값만 계산한 뒤 아무 곳이나 고르는 것이 아니라 **최적 자식의 위치를 기록/재계산**한다. 아래 복원 코드는 동률이면 오른쪽을 택한다. 동률에서 **경로 수**를 물으면 한쪽만 택하지 말고 양쪽 개수를 더한다. 값 계산 $O(N^2)$, 한 행 추가 공간 $O(N)$; 선택 기록은 $O(N^2)$.

학습 예제 [4] / [2, 2] / [1, 3, 1] / [2, 1, 1, 2]의 최대 합은 **10**, 그 합을 만드는 경로는 **4개**. 같은 숫자열이어도 경로 위치가 다르면 따로 세는 조건이다.



최대 합만 계산

```
def trianglepath(T):
    dp = T[-1][:]
    for y in range(len(T) - 2, -1, -1):
        for x in range(y + 1):
            dp[x] = T[y][x] + max(dp[x], dp[x + 1])
    return dp[0]
```

선택 위치로 실제 경로 복원

```
def triangle_solution(T):
    n = len(T)
    dp = T[-1][:]
    path = [[0] * (i + 1) for i in range(n)]
    for y in range(n - 2, -1, -1):
        for x in range(y + 1):
            down, right = dp[x], dp[x + 1]
            path[y][x] = -1 if down > right else 1 if down < right else 0
            dp[x] = T[y][x] + max(down, right)
    x, route = 0, []
    for y in range(n):
        route.append(T[y][x])
        if y < n - 1 and path[y][x] >= 0: x += 1
    return dp[0], route
```

TILING2 · 타일 종류와 기저를 먼저 확인한다

Lec.04 pp. 31–45

2×n 판을 **1×2 도미노만**으로 채우면 $F_n = F_{n-1} + F_{n-2}$. 빈 판을 한 가지로 세는 확장 정의는 $F_0 = F_1 = 1$이며 답은 Fibonacci의 n+1번째와 같다.

2×2 정사각형 타일까지 추가하면 $G_n = G_{n-1} + 2G_{n-2}$, $G_0 = G_1 = 1$, $G_2 = 3$이다. 도미노만 있을 때의 점화식을 그대로 쓰지 않는다. n=3이면 기본3가지 / 추가타일5가지다.

기본 재귀는 지수 시간, 메모·표는 $O(n)$, 두 변수는 **시간** $O(n)$**·추가 공간** $O(1)$이다.

`a, b = b, (a + b) % MOD`는 오른쪽의 **갱신 전 값**으로 동시에 계산한다.

아래 반복 구현은 F(0)=1. 과제의 행렬 버전은 n≥1에서 같고, n=0 반환 규약은 다르다. 행렬 `M = [[1, 1], [1, 0]]`의 n제곱에서 [0][0]을 읽으면 F(n). 빠른 제곱 시간 $O(\log n)$, 재귀 스택 $O(\log n)$.

모듈러는 정수 크기를 제한한다. 이미 큰 a,b를 갖고 `(a % M + b % M) % M`를 한 번 하는 것이 무조건 더 빠르다는 뜻이 아니다. 매 단계 저장값을 M 미만으로 유지해 이후 큰 정수 연산 비용이 커지지 않게 하는 것이 핵심이다.

도미노만 허용한 상수 공간 DP

```
def tiling(n, MOD = 1000000007):
    a, b = 1, 1  # F(0), F(1)
    for _ in range(2, n + 1):
        a, b = b, (a + b) % MOD
    return 1 if n == 0 else b
```

과제와 연결되는 행렬 거듭제곱

```
def tiling_matrix(n, mod = 1000000007):
    def mul(A, B):
        return [[sum(A[i][k] * B[k][j] for k in range(2)) % mod
                 for j in range(2)] for i in range(2)]
    result = [[1, 0], [0, 1]]
    base = [[1, 1], [1, 0]]
    while n:
        if n % 2:
            result = mul(result, base)
        base = mul(base, base)
        n //= 2
    return result[0][0]
```

POLY · 첫 행을 알아야 연결 수가 정해진다

Lec.04 pp. 46–59

P(n, first)는 n칸짜리 **세로로 단조인 폴리오미노** 중 첫 행이 first칸인 경우의 수다. 모든 임의 폴리오미노를 세는 문제로 확대하지 않는다.

다음 행 길이가 second이면 붙이는 가로 위치가 **first+second−1**개다. 아래 수식에서 f는 first, s는 second를 줄인 기호다. $P(n,f)=\sum_{s=1}^{n-f}(f+s-1)P(n-f,s)$. **n == first이면 한 행 하나이므로 1**이다.

답은 첫 행 길이가 정해지지 않았으므로 $\sum_{f=1}^{n}P(n,f)$. $P(4,2)=2P(2,1)+3P(2,2)=5$, 전체 $P(4)=19$.

상태 $O(N^2)$, 전이 $O(N)$ → **시간** $O(N^3)$**, 공간** $O(N^2)$. 과제의 표는 n을 증가시키며 채운다.

명시적 캐시로 점화식 구현

```
def count_poly(n, MOD = 10000000):
    memo = {}
    def poly(n, first):
        if n == first: return 1
        key = (n, first)
        if key not in memo:
            memo[key] = sum((first + second - 1) *
                poly(n - first, second)
                for second in range(1, n - first + 1)) % MOD
        return memo[key]
    return sum(poly(n, f) for f in range(1, n + 1)) % MOD
```

Catalan · 두 부분의 크기를 합치면 n−1

Lec.04 pp. 60–61

기저는 $C_0=1$, 점화식은 $C_n=\sum_{i=0}^{n-1}C_iC_{n-1-i}$이다. 뿌리 하나를 뺀 n−1개를 왼쪽 i개·오른쪽 n−1−i개로 나눈다. 곱은 두 선택을 조합, 합은 모든 나누기를 모은다.

1, 1, 2, 5, 14, 42, …. 모양 개수를 구할 때 C_0을 0으로 두면 한쪽이 빈 경우가 모두 사라진다. **시간** $O(n^2)$**, 추가 공간** $O(n)$. 식의 오른쪽에 n−i를 쓰거나 합 범위에서 n−1을 빼먹지 않는다.

구현 핵심

```
def catalan(n, MOD = 1000000007):
    C = [0] * (n + 1)
    C[0] = 1
    for size in range(1, n + 1):
        for left in range(size):
            C[size] = (C[size] + C[left] * C[size - 1 - left]) % MOD
    return C[n]
```

LEC.05 · 최적해 복원과 순번

PACKING · 최적값과 선택한 물건을 함께 복원한다

Lec.05 pp. 11–23 · Lec.05 pp. 24–30

Fractional은 물건을 쪼갤 수 있어 가치/무게 비율순 탐욕이 성립한다. **0-1**은 통째로 넣거나 빼므로 그 탐욕이 일반적으로 성립하지 않는다.

B(i, c)는 **i번부터, 남은 용량 c**에서 얻는 최대 가치. skip=$B(i+1,c)$, take=$p_i+B(i+1,c-w_i)$ 이며 **w[i]≤c인 경우에만 take**한다. 같은 i라도 c가 다르면 다른 상태다.

복원 시 $B(i,c)=B(i+1,c)$이면 이 구현은 동률에서 건너뛴다. 그 외에는 물건 i를 기록하고 **c −= w[i], i += 1**로 이동한다. 최적값만 맞아도 선택 목록을 복원한 것은 아니다.

시간·캐시 $O(NW)$, 스택 $O(N)$. W는 숫자 값이므로 입력 비트 길이 기준으로는 **pseudo-polynomial(의사다항)**이다. 1행 0-1 배낭은 용량을 **큰 쪽→작은 쪽**으로 갱신해야 같은 물건을 중복 사용하지 않는다.

직접 캐시와 동률 skip 복원

```
def packing(w, p, W):
    n, memo = len(w), {}
    def best(i, c):
        if i == n: return 0
        key = (i, c)
        if key not in memo:
            value = best(i + 1, c)
            if w[i] <= c:
                value = max(value, p[i] + best(i + 1, c - w[i]))
            memo[key] = value
        return memo[key]
    chosen, c = [], W
    for i in range(n):
        if best(i, c) != best(i + 1, c):
            chosen.append(i)
            c -= w[i]
    return best(0, W), chosen
```

LIS · 시작하는 길이와 끝나는 길이를 섞지 않는다

Lec.05 pp. 31–41 · Lec.05 pp. 42–45

엄격한 증가이므로 비교는 <이다. 다음 두 표는 방향이 다르지만 전체 최적값은 같다.

상태	후보 j	계산 순서	[1,3,2,4]의 표
L_i: S_i에서 시작	j>i, S_i<S_j	i 감소	[3,2,2,1]
dp[i]: S_i에서 끝남(실행 과제)	j<i, S_j<S_i	i 증가	[1,2,2,3]

choices[i]는 길이가 아니라 **다음 원소의 인덱스**, −1이면 종료. 시작형은 최댓값 인덱스부터 따라간다. $O(N^2)$ **시간**·$O(N)$ **공간**이다.

빠른 tails[k]는 길이 k+1 증가 부분수열의 **가능한 최소 끝값**. bisect_left로 첫 ≥x 자리를 교체하며 시간 $O(N \log N)$. tails가 최종 LIS 자체라고 단정하지 않는다. [3, 5, 7, 8, 1]에서 tails=[1,5,7,8]은 인덱스 순서상 실제 부분수열이 아니다. 실제 해는 predecessor 등이 더 필요하다.

길이와 choices 복원

```
def lis_solution(S):
    n = len(S)
    if n == 0: return 0, []
    L, choices = [1] * n, [-1] * n
    for i in range(n - 1, -1, -1):
        for j in range(i + 1, n):
            if S[i] < S[j] and L[i] < L[j] + 1:
                L[i], choices[i] = L[j] + 1, j
    start = max(range(n), key = lambda i:L[i])
    length, route = L[start], []
    while start != -1:
        route.append(S[start])
        start = choices[start]
    return length, route
```

tails로 길이만 구하기

```
from bisect import bisect_left
def lis_length(S):
    tails = []
    for x in S:
        k = bisect_left(tails, x)  # 처음으로 x 이상인 위치
        if k == len(tails): tails.append(x)
        else: tails[k] = x
    return len(tails)
```

KLIS · 길이 → 개수 → 값순 묶음 건너뛰기

Lec.05 pp. 46–61

서로 다른 원소, 유효한 1-based k가 전제다. L_i는 그 위치에서 시작하는 최장 길이, C_i는 그 길이를 만드는 수열 수이다. 끝나는 곳은 L_i=1, C_i=1.

다음 후보는 **j>i,** A_i**<**A_j**,** L_i**==**L_j**+1**을 모두 만족해야 한다. 앞의 두 조건만 쓰면 최장이 아닌 증가 부분수열도 세게 된다.

후보를 **값 오름차순**으로 정렬하고 **skip = k−1**을 둔다. C_j≤skip이면 그 묶음을 넘기며 skip−=C[j]. 아니면 그 분기로 간다. 길이가 최대인 시작점 수를 세는 것이 LIS 개수는 아니다.

가상 −∞의 길이 1은 결과에서 뺀다. 아래 구현은 정확한 Python 정수를 쓴다. 개수를 포화시킨다면 $\mathrm{CAP} \geq k$여야 한다. 과제 $\mathrm{CAP} = 2^{31}$과 PDF의 2,000,000,001은 다르며, 후자를 큰 k에 그대로 쓰면 틀릴 수 있다.

길이·개수 DP $O(N^2)$, 아래 매 단계 후보 정렬 복원은 안전 상한 $O(N^2 \log N)$. [5, 1, 6, 4, 3, 2, 8, 7]의 최장 길이3·전체10개·6번째는 **[1,4,8]**이다.

구현 핵심

```
def kth_lis(S, k):  # 서로 다른 값, 유효한 1-based k
    A = [float("-inf")] + S  # 가상 시작점
    n = len(A)
    L, C = [1] * n, [1] * n
    for i in range(n - 1, -1, -1):
        followers = [j for j in range(i + 1, n) if A[i] < A[j]]
        if followers:
            L[i] = 1 + max(L[j] for j in followers)
            C[i] = sum(C[j] for j in followers if L[i] == L[j] + 1)
    if not 1 <= k <= C[0]: raise ValueError("k 범위")
    i, skip, answer = 0, k - 1, []
    while L[i] > 1:
        followers = sorted((A[j], j) for j in range(i + 1, n)
                           if A[i] < A[j] and L[i] == L[j] + 1)
        for value, j in followers:
            if C[j] <= skip: skip -= C[j]
            else:
                answer.append(value)
                i = j
                break
    return answer
```

LEC.06 · 탐욕법과 최소 신장 트리

Greedy · 지금의 선택을 안전하게 확정할 수 있는가

Lec.06 pp. 2–7 · Lec.06 pp. 8–18

Greedy choice property(탐욕적 선택 속성)는 현재 선택을 포함하는 최적해가 적어도 하나 있다는 성질이다. 최적 부분구조만 있다고 아무 국소 선택이나 해도 되는 것은 아니다. **Exchange argument(교환 논증)**는 최적해의 선택을 탐욕 선택으로 바꿔도 나빠지지 않음을 보인다.

MATCHORDER는 **상대 이상(≥)** 레이팅이면 이긴다. 이길 수 있으면 이기는 선수 중 가장 약한 선수를 쓰고, 모두 못 이기면 가장 약한 선수를 버린다. 이분 탐색은 `bisect_left`이다.

Python 정렬 리스트에서 pop은 이동 $O(N)$이므로 아래 총시간은 $O(N^2)$, 정렬·탐색만 보고 $O(N \log N)$이라고 쓰지 않는다. 입력 리스트를 변경하는 구현인지도 확인한다.

구현 핵심

```
from bisect import bisect_left
def matchorder(russian, korean):
    left = sorted(korean)
    wins = 0
    for rating in russian:
        if left[-1] < rating:
            left.pop(0)
        else:
            wins += 1
            left.pop(bisect_left(left, rating))
    return wins
```

STRJOIN · 작은 두 개를 합친 비용을 누적한다

Lec.06 pp. 19–31

두 문자열을 합칠 때 비용이 길이의 합이다. 현재 가장 짧은 두 개 a,b를 꺼내 **a+b를 총비용에 더하고**, 합친 길이를 다시 넣는다. 마지막 문자열의 길이만 반환하면 틀린다.

[2, 3, 6, 8]: 2+3=5, 5+6=11, 8+11=19 → 총비용 **35**. 카드 묶음 응용도 비교 비용을 A+B로 정한 문제에서는 같은 모델이다.

Min-heap으로 **시간** $O(N \log N)$**, 공간** $O(N)$. 합치는 횟수는 N−1이다. Huffman의 가중 경로 길이 최소화와 같은 합치기 구조다.

구현 핵심

```
from heapq import heapify, heappop, heappush
def strjoin(lengths):
    q = lengths[:]
    heapify(q)
    cost = 0
    while len(q) > 1:
        joined = heappop(q) + heappop(q)
        cost += joined
        heappush(q, joined)
    return cost
```

Huffman · 트리 생성, 인코딩, 디코딩을 구분한다

Lec.06 pp. 32–40 · Lec.06 pp. 41–42

출제 예고: 주어진 문자열↔비트열 두 방향. 재귀 코드의 빈칸과 트리 표현 제공 가능성도 언급됐다. "항상 같은 트리·같은 코드·전위순회가 제공된다"는 뜻은 아니다.

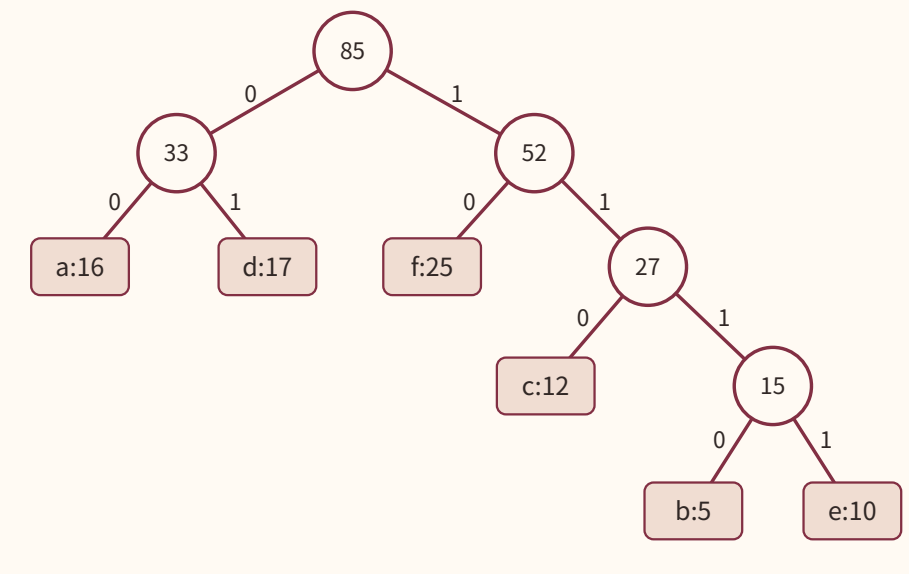
빈도가 작은 **두 트리**를 꺼내 합을 부모 빈도로 한 새 트리를 넣는다. 문자는 잎에만 있으므로 한 문자의 코드가 다른 코드의 접두어가 되지 않는 **prefix-free code**이다. 문자 m개면 잎 m개, 내부 m−1개, 전체 $2m-1$**개**. 비용은 $\sum_c f_c \ell_c$. 수식의 f_c는 문자 c의 빈도, ℓ_c는 그 문자의 코드 길이다.

트리의 왼쪽0·오른쪽1을 따른다. **동일 빈도에서 좌우/동률 선택 규칙이 다르면 코드 비트는 달라질 수 있지만 최적 비용은 같을 수 있다.** 비트를 물으면 문제에 주어진 트리·규칙을 우선한다. 생성 $O(m \log m)$.

강의 트리의 코드표: **a=00, d=01, f=10, c=110, b=1110, e=1111**. `da → 0100, 001110 → ab`. 잎을 읽으면 문자 출력 후 **비트를 더 소비하지 않고 루트 복귀**한다. 불완전한 코드가 남으면 정상 종료가 아니다.

재귀 디코더 빈칸: 내부에서 한 간선 이동은 **i+1**, 잎에서 루트로 다시 시작은 **같은 i**. 인코더는 왼쪽 code+"0", 오른쪽 code+"1", 잎이면 사전에 저장한다.

Preorder=루트→왼쪽→오른쪽. **내부/잎 표식 + 내부 노드 자식이 정확히 둘**이라는 조건이 있으면 그 표현을 재귀적으로 복원할 수 있다. 임의의 일반 이진트리가 전위순회만으로 항상 복원되는 것은 아니다.



우선순위 큐로 트리 만들기

```
from heapq import heapify, heappop, heappush
from itertools import count

class Node:
    def __init__(self, char, freq, left = None, right = None):
        self.char, self.freq = char, freq
        self.left, self.right = left, right

def huffman(pairs):  # 서로 다른 문자, 양의 빈도, 두 문자 이상
    serial = count()
    q = [(f, next(serial), Node(c, f)) for c, f in pairs]
    heapify(q)
    while len(q) > 1:
        fa, _, a = heappop(q)
        fb, _, b = heappop(q)
        parent = Node(None, fa + fb, a, b)
        heappush(q, (fa + fb, next(serial), parent))
    return q[0][2]
```

코드표 생성과 반복 디코더

```
def make_encoder(node, code, encoder):
    if node.char is not None:
        encoder[node.char] = code
    else:
        make_encoder(node.left, code + "0", encoder)
        make_encoder(node.right, code + "1", encoder)

def decode(root, bits):
    node, result = root, []
    for bit in bits:
        if bit not in "01": raise ValueError("비트 오류")
        node = node.left if bit == "0" else node.right
        if node is None: raise ValueError("트리와 불일치")
        if node.char is not None:
            result.append(node.char)
            node = root
    if node is not root: raise ValueError("미완성 코드")
    return "".join(result)
```

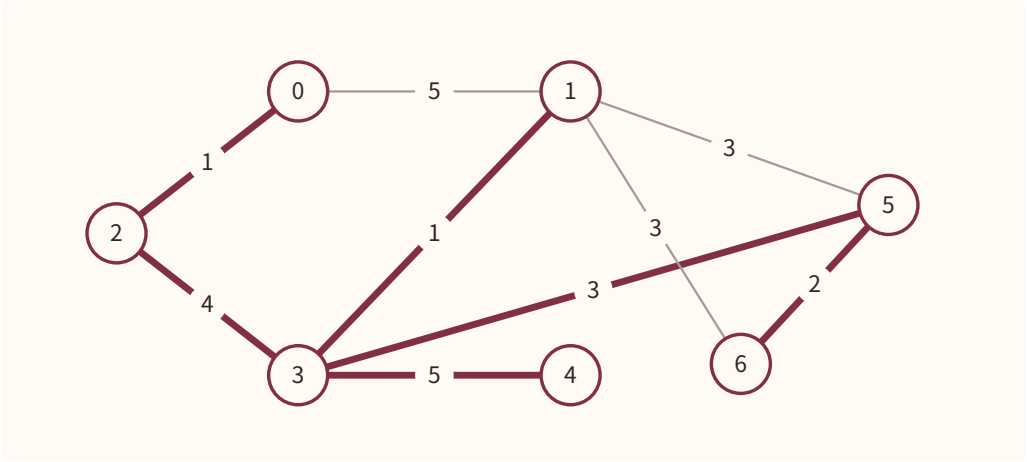
MST·Prim · 경로 합이 아니라 트리에 붙이는 간선 하나

Lec.06 pp. 43–58

연결 무방향 그래프의 **Minimum Spanning Tree**는 모든 정점을 잇는 최소 비용 트리이며 간선은 V−1개다. 최단 경로 트리와 다르다. Prim은 현재 트리 T에서 밖으로 나가는 간선 중 최소 비용을 고른다.

$\text{minweight}_v = \min_{u \in T} w(u, v)$, parent[v]는 그 간선의 T 쪽 끝점이다. 다음 정점은 미방문 중 minweight가 최소인 v. 추가한 u에서 갱신할 때 **w(u,v)**와 비교하며, `minweight[u] + w(u, v)`를 쓰면 Dijkstra식 경로 합이 된다.

표를 물으면 **현재 상태의 표인지, 다음 정점 추가 후 표인지** 확인한다. 동률은 주어진 규칙을 따른다. 정점이 같아도 parent가 유일하지 않을 수 있다. 배열 선택은 $O(V^2 + E)$, 단순 그래프에서 $O(V^2)$. 아래 사전 인접 목록의 공간은 $O(V+E)$, 추가 배열 $O(V)$.



구현 핵심

```
def prim(adj):  # adj[u]: {v: weight}, 연결된 무방향 그래프
    n = len(adj)
    added, parent = [False] * n, [-1] * n
    weight = [float("inf")] * n
    weight[0] = 0
    total, edges = 0, []
    for _ in range(n):
        u = min((v for v in range(n) if not added[v]), key = lambda v:weight[v])
        if weight[u] == float("inf"): raise ValueError("연결되지 않음")
        added[u] = True
        total += weight[u]
        if parent[u] != -1: edges.append((parent[u], u))
        for v, w in adj[u].items():
            if not added[v] and w < weight[v]:
                weight[v], parent[v] = w, u
    return total, edges
```

LAN · 기존 간선 0, 제곱 거리 비교 후 실제 길이 합

Lec.06 pp. 59–61

좌표는 완전 그래프를 만든다. 새 케이블 비용은 Euclidean distance이다. **기존 케이블은 양방향 비용 0**으로 둔다. 제곱 거리로 간선 크기를 비교해도 순서는 같지만 결과에는 **선택한 각 간선에 sqrt를 취해 더한다**.

$\sqrt{\sum_e d_e^2}$와 $\sum_e \sqrt{d_e^2}$는 다르다. 수식의 d_e는 선택한 간선 e의 실제 길이다. 이미 연결된 간선도 Prim의 선택·방문 진행에 포함하며 추가 비용만 0이다. 과제의 `dist = -1` 방문 표식 버전에서 후보는 **dist≥0**이어야 0원 간선을 버리지 않는다.

A=(0,0), B=(3,0), C=(3,4), A–B가 기존 연결이면 새 비용은 B–C의 **4**이다. 거리 행렬을 쓰는 아래 구현은 **시간·공간** $O(V^2)$. n≥1을 전제한다.

구현 핵심

```
import math
def lan(points, installed):
    n = len(points)
    W = [[(x - a) ** 2 + (y - b) ** 2 for a, b in points] for x, y in points]
    for u, v in installed: W[u][v] = W[v][u] = 0
    dist, added = [float("inf")] * n, [False] * n
    dist[0] = 0
    total = 0.0
    for _ in range(n):
        u = min((v for v in range(n) if not added[v]), key = lambda v:dist[v])
        total += math.sqrt(dist[u])
        added[u] = True
        for v in range(n):
            if not added[v]: dist[v] = min(dist[v], W[u][v])
    return total
```

시험장 들어가기 전 마지막 확인

- 유한성은 종료, 정확성은 원하는 답, 복잡도는 자원이다.
- NP는 빠른 검증이며 P도 포함한다. $P=NP$는 미해결이다.
- 누적합의 양끝 포함/끝 제외 규약과 길이 식은 같은 규약을 써야 한다.
- MAXSUM 강의 규약은 빈 구간을 허용해 전부 음수이면 0이다.
- 4분기 깊이 d 트리의 리프는 4^d, 전체 노드는 $\sum_{k=0}^{d} 4^k$, 루트 포함 호출 스택은 d+1이다.
- 배치한 공유 상태는 재귀 호출이 끝나면 복구한다.
- Master의 a는 호출 수, b는 입력 축소 비율, n^k는 재귀 밖 비용이다.
- QUADTREE는 자식을 원래 순서로 재귀해 읽고 3→4→1→2로 결합한다.
- 이항계수·0-1 배낭은 오른쪽→왼쪽, 아래→위 삼각형 한 행 DP는 왼쪽→오른쪽이다.
- 최적값·최적해·최적해 개수·k번째 최적해는 서로 다른 결과다.
- LIS의 시작형/종료형 방향과 strict < 조건을 확인한다. tails 자체는 복원된 LIS가 아니다.
- KLIS는 최장 길이 조건을 통과한 후보만 값순으로 정렬하고 skip=k−1만큼 건너뛴다.
- Huffman의 잎에서 문자를 출력하고 루트로 복귀할 때 새 비트를 소비하지 않는다.
- Prim 갱신은 간선 하나의 비용이며 경로 누적합이 아니다. LAN의 기존 연결은 비용 0이다.

출제 예고 근거: CLOCKSYNC SST(2차시 44분대), QUADTREE 뒤집기(사용자 청취 확인), Master 코드 제공 방식(3차시 35–39분대), Huffman 인코딩·디코딩/빈칸(6차시 1:01–1:04). Catalan과 CLOCKSYNC 개선 설명을 시험 제외·추가 출제 확정으로 바꾸지 않는다. QUADTREE 인코딩/디코딩은 보충 구현 권고다.